

Penetration Testing Report on OWASP Juice Shop

Author: Barbara Bielecka

Date: June 2026

**Tools Used: Kali Linux, Burp Suite Community Edition, Firefox,
OWASP Juice Shop Score Board**

Executive Summary

This project presents the results of a security assessment conducted on OWASP Juice Shop, a deliberately vulnerable web application designed for security training. The objective was to identify and analyze common web application vulnerabilities using industry-standard penetration testing techniques and tools. The assessment included reconnaissance, HTTP traffic analysis, vulnerability identification, exploitation of selected weaknesses, and documentation of findings.

Several high-risk and medium-risk vulnerabilities were identified, demonstrating the impact of weaknesses in authentication, input validation, access control, and configuration management. The project demonstrates practical skills in vulnerability assessment, risk analysis, and security reporting.

Introduction

OWASP Juice Shop is a deliberately vulnerable web application designed for security training and penetration testing practice. It contains vulnerabilities based on the OWASP Top 10 and other real-world security issues, making it a useful platform for hands-on learning.

The objective of this project was to identify, analyze, and document security vulnerabilities within the application using common penetration testing methodologies and tools. The assessment demonstrates practical web application security skills, including vulnerability discovery, exploitation, impact assessment, and reporting.

Project Description

Project Overview

This project documents a penetration testing engagement performed against OWASP Juice Shop. The assessment focused on identifying common web application vulnerabilities and understanding their potential impact on confidentiality, integrity, and availability.

Scope

- OWASP Juice Shop web application.
- Authentication and authorization mechanisms.
- User input validation.

- Session management.
- Client-side and server-side security controls.

Tools Used

- Burp Suite
- Kali Linux
- Browser Developer Tools
- OWASP Juice Shop Score Board

Key Activities

- Application reconnaissance.
- HTTP request and response analysis.
- Vulnerability identification and exploitation.
- Evidence collection using screenshots.
- Risk assessment.
- Security recommendations.

Skills Demonstrated

- Web Application Penetration Testing
- Vulnerability Assessment
- OWASP Top 10 Awareness
- Burp Suite Usage
- Security Reporting
- Technical Documentation

Testing Environment

Component	Details
Operating System	Kali Linux
Target Application	OWASP Juice Shop

Component	Details
Testing Tool	Burp Suite Community Edition
Browser	Firefox
Methodology	Manual Penetration Testing

1. Introduction

This project presents the results of a security assessment conducted on OWASP Juice Shop, a deliberately vulnerable web application designed for security training. The objective was to identify and analyze common web application vulnerabilities using industry-standard penetration testing techniques and tools.

The assessment included reconnaissance, HTTP traffic analysis, vulnerability identification, and exploitation of selected security weaknesses. Several vulnerabilities were successfully identified, demonstrating risks related to authentication, input validation, and access control.

The findings highlight the importance of secure development practices and provide recommendations to mitigate the identified security issues. This project demonstrates practical skills in web application penetration testing, vulnerability assessment, and security reporting.

2. Methodology

- Reconnaissance
- Application mapping
- Vulnerability assessment
- Exploitation
- Documentation

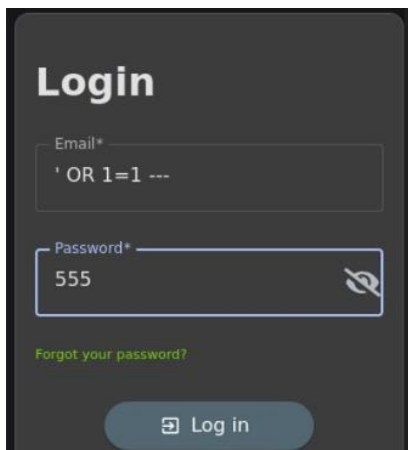
3. Findings

Finding #1: SQL Injection

Description :

SQL Injection (SQLi) is a web security vulnerability that allows an attacker to interfere with the queries an application makes to its database. This can enable attackers to view, modify, or delete data they are not normally authorized to access, potentially exposing sensitive information such as passwords, credit card details, and personal user data.

Evidence :



Impact

It has a significant impact on juice shop security.

- Authentication bypass through malicious SQL input in the username field.
- Extraction of sensitive data from the database, including user credentials and customer information.
- Privilege escalation, potentially leading to administrative access.
- Denial of service caused by malicious queries, timeouts, or resource exhaustion.

Risk Level

High

Remediation

1. Use parameterized queries.
2. Validate and sanitize input.
3. Use stored procedures where appropriate.
4. Apply least privilege principles.

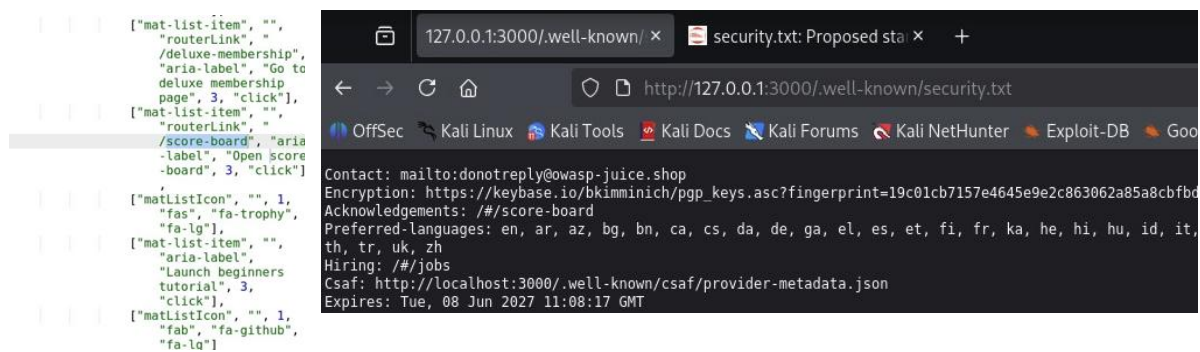
5. Improve error handling.
6. Deploy a web application firewall.
7. Perform regular code review.

Finding #2: Misscelaneous

Description :

This category includes a broad range of issues such as exposed Docker logs, insecure JWT implementation, missing CORS validation, and inadequate file upload filtering.

Evidence



Impact

- Reduced security of the testing environment.
- Exposure of infrastructure or logging information.
- Configuration weaknesses that may support further attacks.
- Lowered effectiveness of the training environment.

Risk Level

Medium

Remediation

1. Apply configuration hardening.
2. Improve security header protection.
3. Strengthen logging practices.
4. Enable continuous monitoring.

5. Improve container security.
6. Harden the infrastructure.

Finding #3: Security Misconfiguration

Description :

Security misconfiguration occurs when applications, servers, or frameworks are deployed with insecure defaults or improper settings. These vulnerabilities can allow attackers to access unauthorized resources, extract sensitive data, or escalate privileges.

Evidence

The screenshot displays a web application security tool interface. On the left, a list of intercepted requests is shown. The selected request is a GET to `/rest/kali` with a Bearer token. The token is a long alphanumeric string. The response is a 200 OK. The tool also shows a 'Complaint' form with fields for Customer (kali@owasp.com) and Message (hello kali). The form has a character limit of 160 and a 'Submit' button.

Impact

- Outdated dependency versions.
- Forgotten admin panel.
- Exposed administrative routes without authentication.
- Verbose error messages revealing database schema information.
- Default admin credentials still enabled.
- Missing Content Security Policy headers.

- Exposed Docker configuration files.

Risk Level

Medium

Remediation

1. Improve error handling.
2. Remove deprecated interfaces.
3. Implement security headers.
4. Maintain dependency management and patching.

Finding #4: Improper Input Validation

Description :

This vulnerability occurs when an application fails to properly validate user-supplied data before processing it.

Evidence

1.

User Registration

Email*
juice@shop.com

Password*
●●●●●

ⓘ Password must be 5-40 characters long. 5/20
Repeat Password*
●●●●●●●●

9/40

Show password advice

Security Question *
Mother's maiden name?

ⓘ This cannot be changed later!
Answer*
juiceshop

Register

Already a customer?

Login

Email*

Password*

Forgot your password?

Log in

☐ Remember me

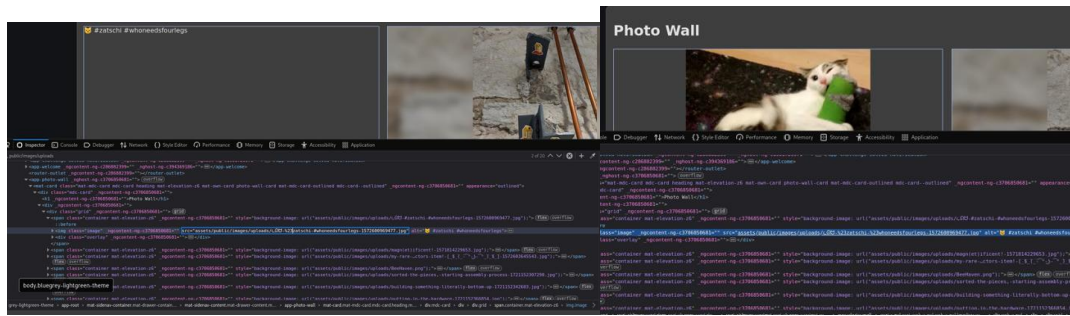
or

Log in with Google

Not yet a customer?

You successfully solved a challenge: Repetitive Registration (Follow the DRY principle while registering a user.)

2.



Impact

- Attack vectors such as SQL injection, XSS, and file upload bypass.
- Complete account takeover.
- Privilege escalation.
- Data exfiltration.
- Server compromise.
- Business logic exploitation.

Risk Level

High

Remediation

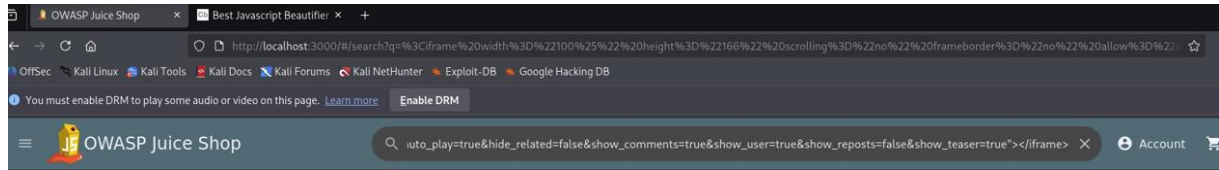
1. Apply input sanitization.
2. Use parameterized queries.
3. Implement rate limiting.
4. Enable audit logging.

Finding #5: Cross-Site Scripting (XSS)

Description :

Cross-Site Scripting (XSS) is a web vulnerability that allows an attacker to inject malicious scripts into pages viewed by other users. The injected scripts execute in victims' browsers with their privileges.

Evidence



Impact

- Session hijacking through stolen cookies.
- Keylogging.
- Phishing.
- Defacement.
- CSRF token theft.
- Account takeover.

Risk Level

High

Remediation

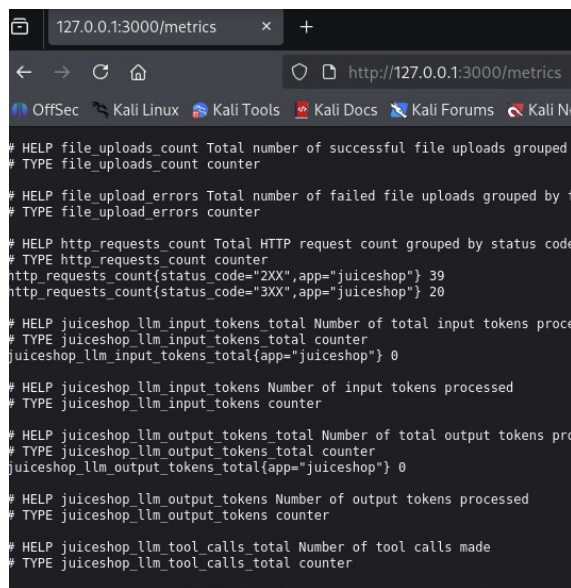
1. Use output encoding.
2. Apply a strong Content Security Policy.
3. Validate user input.
4. Use framework-level protection.
5. Sanitize DOM content.
6. Follow secure coding practices.

Finding #6: Observability Failures

Description :

This vulnerability occurs when monitoring systems inadequately track application behavior, creating blind spots in security detection.

Evidence



```
# HELP file_uploads_count Total number of successful file uploads grouped by status code
# TYPE file_uploads_count counter

# HELP file_upload_errors Total number of failed file uploads grouped by status code
# TYPE file_upload_errors counter

# HELP http_requests_count Total HTTP request count grouped by status code
# TYPE http_requests_count counter
http_requests_count{status_code="2XX",app="juiceshop"} 39
http_requests_count{status_code="3XX",app="juiceshop"} 20

# HELP juiceshop_llm_input_tokens_total Number of total input tokens processed
# TYPE juiceshop_llm_input_tokens_total counter
juiceshop_llm_input_tokens_total{app="juiceshop"} 0

# HELP juiceshop_llm_input_tokens Number of input tokens processed
# TYPE juiceshop_llm_input_tokens counter

# HELP juiceshop_llm_output_tokens_total Number of total output tokens processed
# TYPE juiceshop_llm_output_tokens_total counter
juiceshop_llm_output_tokens_total{app="juiceshop"} 0

# HELP juiceshop_llm_output_tokens Number of output tokens processed
# TYPE juiceshop_llm_output_tokens counter

# HELP juiceshop_llm_tool_calls_total Number of tool calls made
# TYPE juiceshop_llm_tool_calls_total counter
```

Impact

- Privilege escalation without proper tracking of role changes.
- Data exfiltration without monitoring of large transfers.
- Session hijacking without detection of cookie theft.
- API abuse without tracking of exceeded rate limits.

Risk Level

High

Remediation

1. Implement structured logging.
2. Define security metrics.
3. Improve distributed tracing.
4. Maintain audit trails.
5. Add compliance tracking.
6. Enable real-time monitoring

The screenshot displays the Juice Shop application interface. At the top, a green banner contains the text: "You successfully solved a challenge: Admin Section (Access the administration section of the store.)" with a link "Jump to related coding challenge".

The main content area is divided into two sections:

- Administration:** A table titled "Registered Users" lists ten users with their email addresses and a status icon (eye or key).
- Customer Feedback:** A table titled "Customer Feedback" lists ten feedback entries with their IDs, text, star ratings, and a status icon (eye or key).

Registered Users	
admin@juice-sh.op	👁
jim@juice-sh.op	👁
bender@juice-sh.op	👁
bjoern.kimminich@gmail.com	👁
ciso@juice-sh.op	👁
support@juice-sh.op	👁
morty@juice-sh.op	👁
mc.safesearch@juice-sh.op	👁
j12934@juice-sh.op	👁
wurstbrot@juice-sh.op	👁

Customer Feedback	
1	I love this shop! Best products in town! Highly recommended! (***in@juice-sh.op) ★★★★★
2	Great shop! Awesome service! (***@juice-sh.op) ★★★★★
3	Nothing useful available here! (***der@juice-sh.op) ★
21	Please send me the juicy chatbot NFT in my wallet at /juicy-nft : purpose betray marriage blame crunch monitor spin slide donate sport lift clutch" (***ereum@juice-sh.op) ★
	Incompetent customer support! Can't even upload photo of broken purchase! Support Team: Sorry, only order confirmation PDFs can be attached to complaints! (anonymous) ★★
	This is the store for awesome stuff of all kinds! (anonymous) ★★★★★
	Never gonna buy anywhere else from now on! Thanks for the great service! (anonymous) ★★★★★

Impact

- Unauthorized access to administrative functions.[P](#)
- Ability to modify application configuration.
- Potential for further privilege escalation.
- Exposure of sensitive data through admin APIs.
- Risk of denial-of-service attacks through admin endpoints.

Risk Level

High

Remediation

1. Enforce strict authentication for all admin endpoints.
2. Add authorization checks for each admin action.
3. Use role-based access control (RBAC).
4. Log all administrative activity.
5. Apply input validation to reduce injection risks.

Finding #8: Sensitive Data Exposure

Description :

Sensitive Data Exposure occurs when an application fails to properly protect confidential information, exposing it to unauthorized users or attackers.

Evidence

The evidence consists of two screenshots. The top-left screenshot shows a terminal window running a DirB brute-force attack on an FTP server at 127.0.0.1:3000. The terminal output shows the attack parameters, the list of generated words (4612), and the results of the scan, including several discovered files and directories. The top-right screenshot shows a web browser displaying the contents of the file `http://127.0.0.1:3000/ftp/acquisitions.md`. The file contains a header "# Planned Acquisitions", a warning "This document is confidential! Do not distribute!", and a paragraph of Lorem Ipsum text. The bottom screenshot shows a web browser displaying the contents of the file `http://127.0.0.1:3000/ftp/`, which is a directory listing of the FTP root. The listing shows various files and directories, including `quarantine`, `coupons_2013.md.bak`, `incident-support.kdbx`, `package.json.bak`, `acquisitions.md`, `eastere.gg`, `legal.md`, `suspicious_errors.yml`, `announcement_encrypted.m`, `encrypt.pyc`, and `package-lock.json.bak`.

```
(kali@kali)-[~]
$ dirb http://127.0.0.1:3000 -f

DIRB v2.22
By The Dark Raver

START_TIME: Tue Jun 2 06:27:22 2026
URL_BASE: http://127.0.0.1:3000/
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt
OPTION: Fine tuning of NOT_FOUND detection

GENERATED WORDS: 4612

--- Scanning URL: http://127.0.0.1:3000/ ---
+ http://127.0.0.1:3000/assets (CODE:301|SIZE:156)
+ http://127.0.0.1:3000/ftp (CODE:200|SIZE:941)
+ http://127.0.0.1:3000/media (CODE:301|SIZE:155)
+ http://127.0.0.1:3000/profile (CODE:500|SIZE:1043)
+ http://127.0.0.1:3000/promotion (CODE:200|SIZE:671)
+ http://127.0.0.1:3000/redirect (CODE:500|SIZE:3113)
+ http://127.0.0.1:3000/robots.txt (CODE:200|SIZE:3)
+ http://127.0.0.1:3000/video (CODE:200|SIZE:263788)
+ http://127.0.0.1:3000/Video (CODE:200|SIZE:263788)

END_TIME: Tue Jun 2 06:27:53 2026
DOWNLOADED: 4612 - FOUND: 9
```

Planned Acquisitions

> This document is confidential! Do not distribute!

Our company plans to acquire several competitors within the next year. This will have a significant stock market impact as we will elaborate in detail in the following paragraph:

Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet.

Our shareholders will be excited. It's true. No fake news.

~ / ftp

quarantine	acquisitions.md	announcement_encrypted.m
coupons_2013.md.bak	eastere.gg	encrypt.pyc
incident-support.kdbx	legal.md	package-lock.json.bak
package.json.bak	suspicious_errors.yml	

Impact

- Exposure of internal documents, source code, or sensitive configurations.
- Potential disclosure of intellectual property.
- Information leakage about internal architecture.
- Possible credential extraction from configuration files.
- Business logic exposure that may support targeted attacks.

Risk Level

Medium-High

Remediation

2. Implement proper authentication for document access.
3. Use encrypted storage for sensitive files.
4. Apply role-based access control.
5. Validate file paths to prevent directory traversal.
6. Monitor access to sensitive documents.
7. Apply least privilege principles.

6. Risk Summary

Vulnerability	Severity
SQL Injection	High
Miscellaneous	Medium
Security Misconfiguration	Medium
Improper Input Validation	High
XSS	High
Observability Failures	High
Broken Access Control	High
Sensitive Data Exposure	Medium-High

7. Recommendations

- Implement parameterized queries and strict input validation to mitigate SQL injection and related injection risks.
- Apply output encoding, DOM sanitization, and a strong Content Security Policy to reduce XSS impact.
- Enforce access control checks on every sensitive route and admin function, using role-based access control and authentication for protected actions.
- Remove exposed debug, legacy, and admin endpoints; disable default credentials; and harden configuration to reduce misconfiguration exposure.
- Improve logging, monitoring, and audit trails so suspicious activity such as privilege changes, abuse, or data exfiltration can be detected faster.

- Protect sensitive data with least privilege, encrypted storage, and tighter file and document access controls.

8. Skills Demonstrated

- Web application penetration testing using manual testing techniques.
- Vulnerability identification and analysis across common OWASP Top 10 issues.
- Burp Suite traffic interception, request modification, and parameter testing.
- Risk assessment based on the potential impact of discovered weaknesses.
- Security reporting and documentation of evidence, findings, and remediation guidance.
- Understanding of authentication, authorization, session handling, and input validation weaknesses.

9. Conclusion

The security assessment of OWASP Juice Shop revealed multiple high-risk and medium-risk vulnerabilities, including SQL injection, XSS, broken access control, and improper input validation. These issues demonstrate the real-world impact of weak application security controls.

Overall, the project confirms practical skills in penetration testing, vulnerability analysis, and technical reporting.